Code Coverage Report

Railway Reservation System - Team 15

1. Purpose and scope

This report gives the statement (line) and function coverage achieved by the automated test suite. Coverage covers the core library in src/ (11 files). src/main.cpp, the interactive CLI, is excluded because it is verified through manual system testing.

Field	Value
Project	Railway Reservation System (RRS)
Team	Team 15 (TEAM15-RRS)
Repository	TEAM15-RRS/se-miniproject-railway
Code version tested	Branch main, commit 7ff1f9c (3 Nov 2026)
Report date	11 November 2026

2. Tools and method

- GCC 13.3.0 with --coverage -O0 (gcov instrumentation), Debug build.
- All three CTest targets were run: unit_tests, integration_tests, search_performance_test.
- Report generated with gcovr 8.8: gcovr -r . --filter src/ --exclude src/main.cpp
- The full HTML report is delivered with this report in the coverage-html folder.

3. Overall result

Metric	Covered / Total	Percentage
Line coverage	848 / 927	91.5%
Function coverage	87 / 89	97.8%

4. Coverage by file

File	Lines	Executed	Line coverage
src/admin.cpp	188	185	98%
src/auth.cpp	58	56	96%
src/booking.cpp	90	89	98%
src/cancel.cpp	63	60	95%
src/inventory.cpp	22	22	100%
src/payment.cpp	33	32	97%
src/search.cpp	28	28	100%
src/sha256.cpp	31	31	100%
src/storage.cpp	308	250	81%
src/types.cpp	34	29	85%

File	Lines	Executed	Line coverage
src/util.cpp	72	66	91%
Total	927	848	91.5%

5. Analysis of uncovered code

About 8.5% of lines (79 lines) are not executed. They fall into these groups:

- storage.cpp (81%, 58 lines): error and rollback paths of backup/restore, such as a missing data file, a failed staging copy, a failed directory creation and the rollback after a failed replacement. These need simulated file-system failures to trigger.
- util.cpp: currentMinutes(), which reads the real system clock. Tests use an injected fixed clock so results are repeatable.
- admin.cpp, cancel.cpp, auth.cpp, booking.cpp, payment.cpp: a few defensive guards (for example a train that no longer exists for a stored ticket, or deactivating an already inactive train).
- types.cpp: a small number of formatting helpers for unusual values.

All core business logic (search, booking, inventory, cancellation, payment, hashing) is at 95% or higher.

6. Conclusion

The test suite executes 91.5% of the core code and 97.8% of its functions. The remaining gaps are error-handling paths and a wall-clock helper, which are low risk. Adding tests that simulate file-system failures would raise storage.cpp coverage.

Branch Coverage Report

Railway Reservation System - Team 15

1. Purpose and scope

This report gives the branch coverage achieved by the automated test suite on the core library (src/, excluding main.cpp). A branch is one outcome of a decision, such as the true or false side .

Field	Value
Project	Railway Reservation System (RRS)
Team	Team 15 (TEAM15-RRS)
Repository	TEAM15-RRS/se-miniproject-railway
Code version tested	Branch main, commit 7ff1f9c (3 Nov 2026)
Report date	11 November 2026

2. Tools and method

- GCC 13.3.0 with --coverage -O0, Debug build; all three CTest targets run.
- Report generated with gcovr 8.8 using --exclude-throw-branches and --exclude-unreachable-branches.
- These options remove compiler-generated exception-handling branches (hidden paths in std::string, std::vector and similar), which source-level tests cannot reach. Without them gcov reports 1,365 of 2,450 branches (55%), which would understate the real coverage.
- The HTML report with branch detail is delivered in the coverage-html folder.

3. Overall result

Metric	Covered / Total	Percentage
Branch coverage (source-level)	1,286 / 1,590	80.9%
Branch coverage (raw gcov, including exception branches)	1,365 / 2,450	55.7%

The source-level figure (80.9%) is the one used for this project.

4. Branch coverage by file

File	Branches	Taken	Branch coverage
src/admin.cpp	206	183	88%
src/auth.cpp	84	81	96%
src/booking.cpp	193	182	94%
src/cancel.cpp	81	71	87%
src/inventory.cpp	34	31	91%
src/payment.cpp	38	38	100%
src/search.cpp	61	61	100%

File	Branches	Taken	Branch coverage
src/sha256.cpp	21	21	100%
src/storage.cpp	698	469	67%
src/types.cpp	44	36	81%
src/util.cpp	130	113	86%
Total	1590	1286	80.9%

5. Analysis of uncovered branches

1.storage.cpp (67%, 229 uncovered): most are the failure side of file operations in backup/restore (missing files, copy failures, rollback). These need injected file-system errors to test.

2.admin.cpp (88%) and cancel.cpp (87%): the untaken side of defensive checks, such as a ticket whose train record is missing or an already-inactive train.

3.Types.cpp (81%) and util.cpp (86%): rare formatting cases and the wall-clock helper.

Booking, authentication, payment, search and hashing are at 94% to 100%, so the main decision logic of the system is well tested, including the boundary cases (age 0/1/120/121, passengers 0/6/7, username and password length limits, refund slabs at 24 h and 12 h).

6. Recommendations

- Add tests with a read-only or missing data directory to cover the storage error paths.
- Add a test for a stored ticket whose train was removed, to cover the defensive guards in admin and cancel.

7. Conclusion

The suite takes 80.9% of source-level branches, with the core business logic above 90%. The gaps are mainly error handling in backup and restore, which is low risk for the intended use.